

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Coherence (Part 4)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan, Natalie Enright Jerger and Mikko Lipasti

Last Lecture

- Coherence misses
- Directory-based coherence – MSI
- Transient states

This Lecture

- Directory-based coherence – MSI
- Directory structure
- Directory-based coherence – MESI

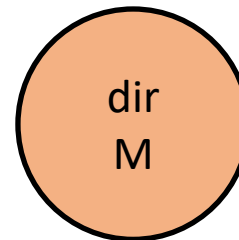
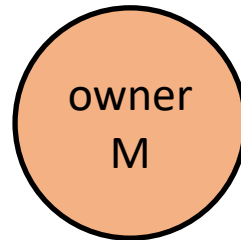
Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

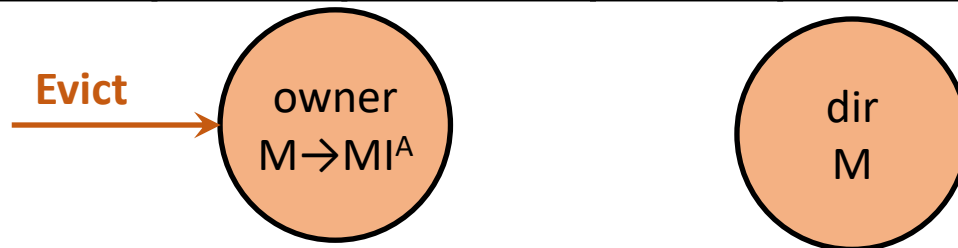


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

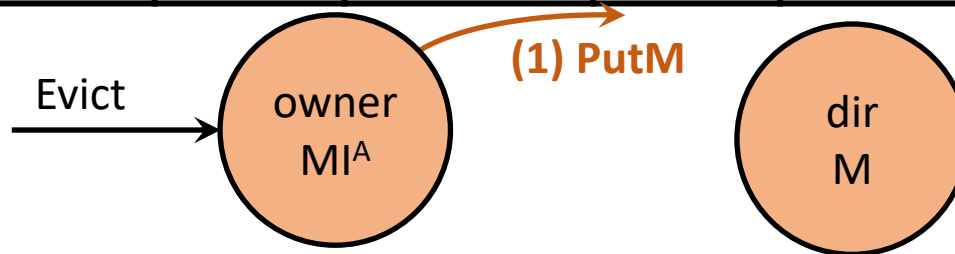


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

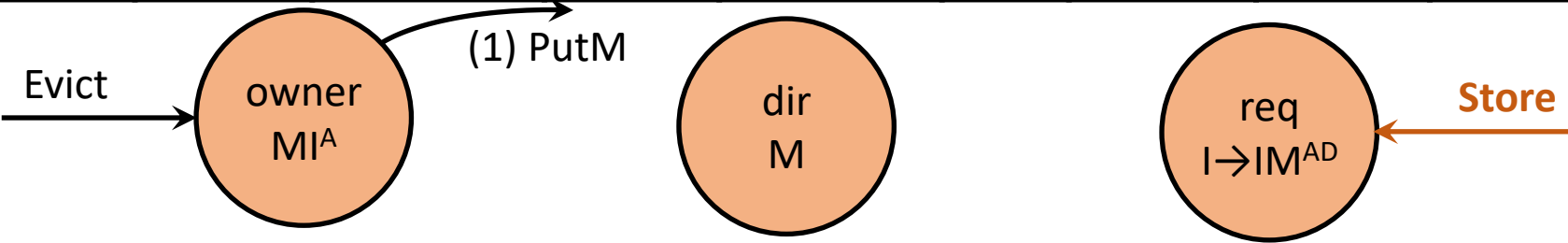


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

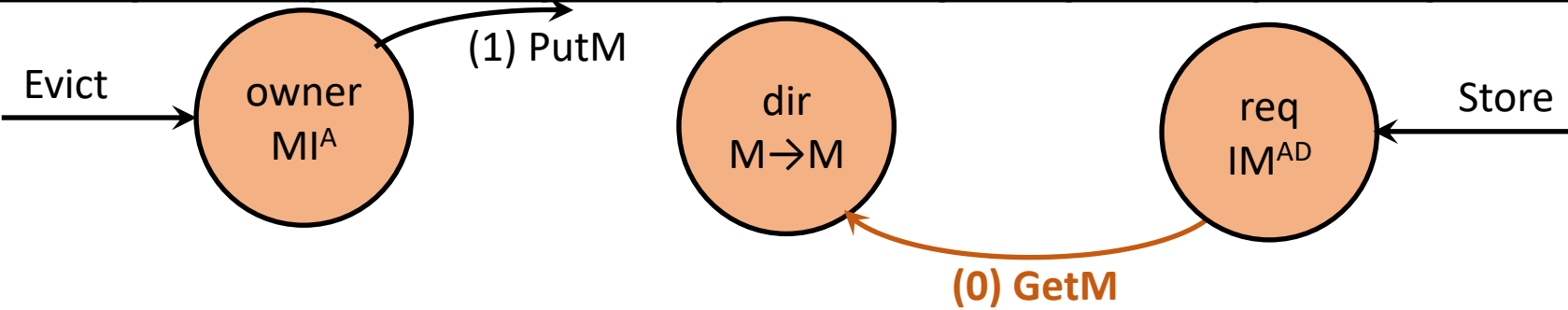


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

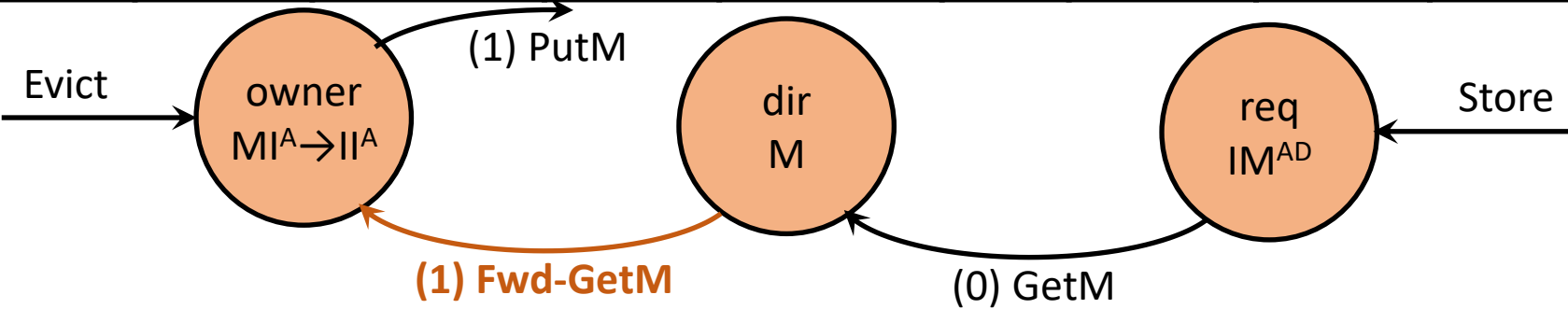


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

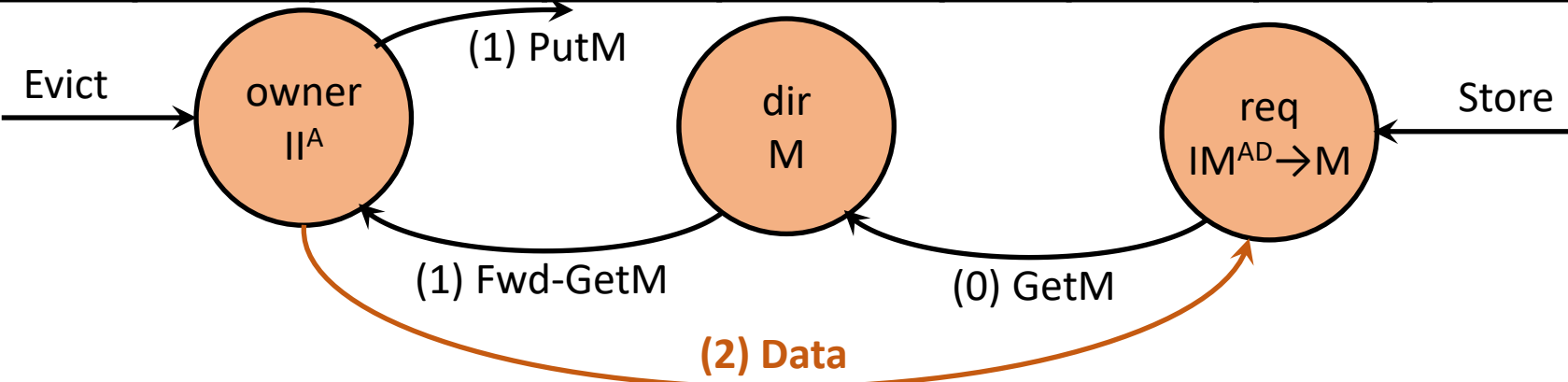


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

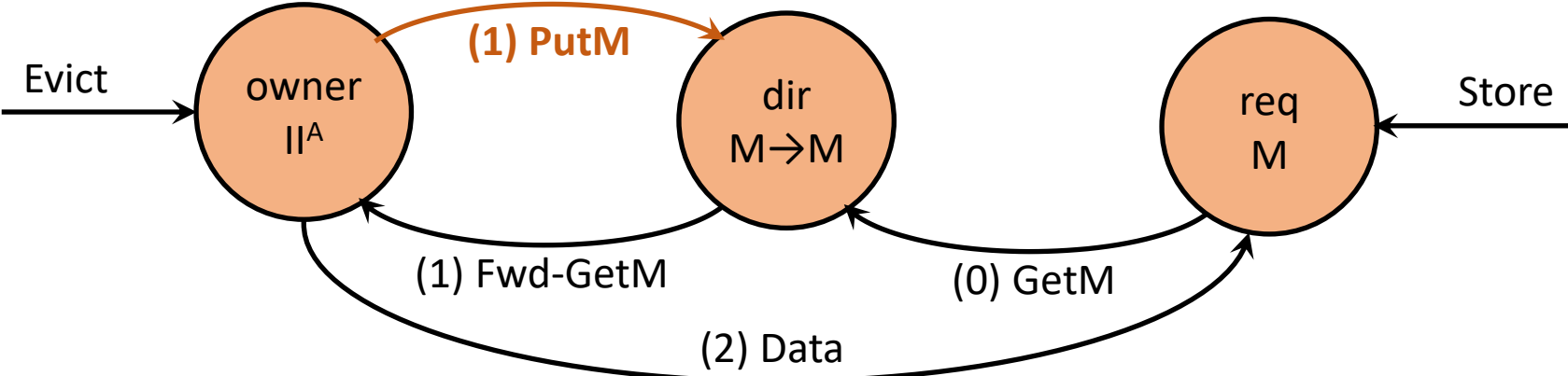


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

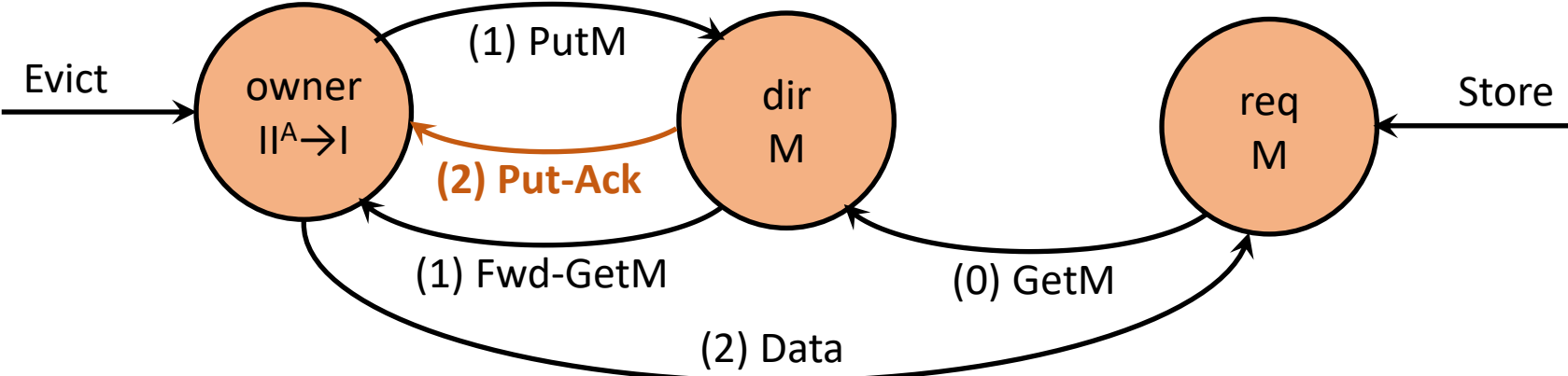


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

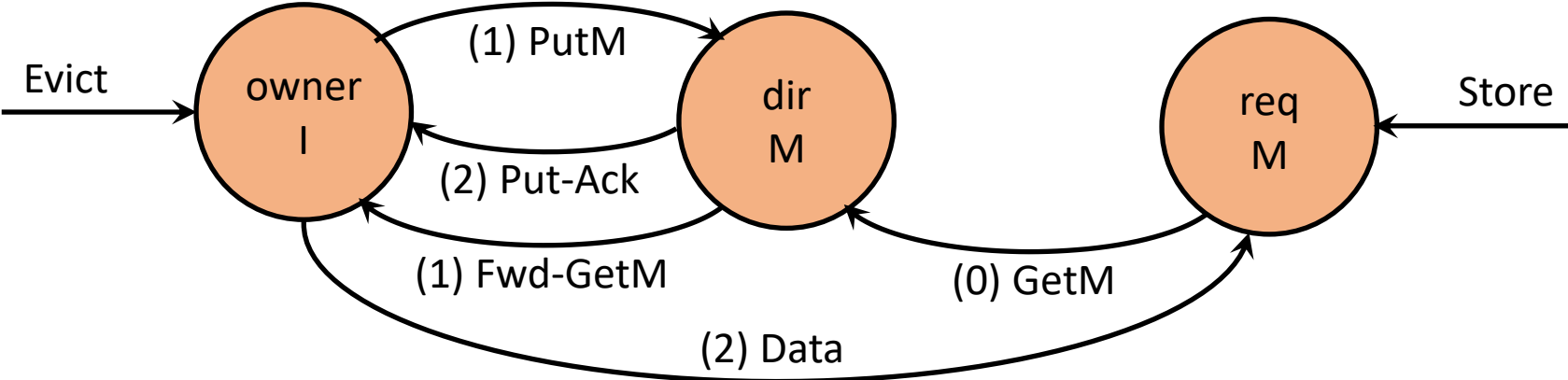


Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory-Based Coherence

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					



Directory Controller

Current State	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	

Directory Structure – Sharers

bit vectors

state	owner	sharers
owned	8	0000011100001100000000

false positives are okay



Directory Structure – Sharers

bit vectors

state	owner	sharers
owned	8	0000011100001100000000

can be *fine-grained*:
one bit per cache

can be *coarse-grained*:
one bit per cluster of caches
(set if at least one is sharer)

Directory Structure – Sharers

bit vectors

state	owner	sharers
owned	8	0000011100001100000000

limited pointers

state	owner	sharers
owned	8	5; 6; 7; 12; 13; 34;

Directory Structure – Sharers

bit vectors

state	owner	sharers
owned	8	0000011100001100000000

limited pointers

state	owner	sharers
owned	8	5; 6; 7; 12; 13; 34;

if full:

- 1) force invalidation; or
- 2) resort to broadcast

Directory Structure – Sharers

bit vectors

state	owner	sharers
owned	8	0000011100001100000000

limited pointers

state	owner	sharers
owned	8	5; 6; 7; 12; 13; 34;

linked list pointers

state	owner	head	sharers
			5,next;
			34,next;
owned	8	60	
			7, next;
			6,next;
			13,next;
			12,next;

Directory Structure – Sharers

bit vectors

state	owner	sharers
owned	8	0000011100001100000000

limited pointers

state	owner	sharers
owned	8	5; 6; 7; 12; 13; 34;

linked list pointers

state	owner	head	sharers
			5,next;
			34,next;
owned	8	60	
			7, next;
			6,next;
			13,next;
			12,next;

can also store *prev* to
simplify removing sharers

Directory Structure – Sharers

bit vectors

state	owner	sharers
owned	8	0000011100001100000000

bloom filters

state	owner	sharers
		0100110101010100101010
		1111100010001000001100
		0000110000000011000000
owned	8	0000011100001100000000
		0000111001010001100000
		1111110000000000000001
		0000000100010000000000
		0000000001010100000001
		1001000000100000010000
		0000100000001000010000
		0000001000100000011000
		1111000100000101000000
		1100000110000001100001
		0001010000010010000100

hash(core, addr)



Directory Structure – Sparse Directory

non-sparse directory

state	owner	sharers
owned	8	0000011100001100000000

table of all addresses

vs.

sparse directory

tag	LRU	state	owner	sharers
0x1234	1	owned	8	0000011100001100000000

cache of only valid addresses



Directory Structure – Sparse Directory

non-sparse directory

state	owner	sharers
owned	8	0000011100001100000000

table of all addresses

➤ lower utilization;
not many unique blocks in private caches

sparse directory

tag	LRU	state	owner	sharers
0x1234	1	owned	8	0000011100001100000000

vs.

cache of only valid addresses

➤ needs high associativity;
recalls (invalidates) sharers upon eviction



Directory Structure – Inclusive Caches

sparse directory					inclusive cache
tag	LRU	state	owner	sharers	data

if inclusive cache, can embed
directory in cache tag array

Directory-Based Coherence – Requirements

Network ordering:

All messages from source X to destination Y are assumed to be ordered.

Deadlock avoidance:

Messages of different types should traverse through different virtual networks.

Starvation avoidance:

Network should provide fair arbitration to prevent starvation.

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

At most how many transient states might you need from I to M?

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

At most how many transient states might you need from I to M? 2

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

At most how many transient states might you need from M to I?

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

At most how many transient states might you need from M to I? 3

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

At most how many transient states might you need from S to M?

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

At most how many transient states might you need from S to M? 3

Directory-Based Coherence

	initiating my transaction			someone else's transaction			responses to my transaction					
Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Directory-Based Coherence

	initiating my transaction			someone else's transaction			responses to my transaction					
Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

My transaction: ____ → ____.
 Their transaction: ____ → ____.
 ____ transaction got to dir first.

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

My transaction: S → M .
 Their transaction: ____ → ____ .
 ____ transaction got to dir first.

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

My transaction: S → M.
 Their transaction: I/S → M.
 transaction got to dir first.

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

My transaction: S → M.
 Their transaction: I/S → M.
Their transaction got to dir first.

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

My transaction: ____ → ____.
 Their transaction: ____ → ____.
 ____ transaction got to dir first.

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

My transaction: I/S → M.
 Their transaction: ____ → ____.
 ____ transaction got to dir first.

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

My transaction: I/S → M.
 Their transaction: I → S.
 transaction got to dir first.

Directory-Based Coherence

Cache Controller

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

My transaction: I/S → M.
 Their transaction: I → S.
My transaction got to dir first.

Directory-Based Coherence

	initiating my transaction			someone else's transaction			responses to my transaction					
Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Can only stall if other transaction got to dir after me

Directory-Based Coherence

Cache Controller

why is Inv possible in IS^D but not IM^{AD}?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Directory-Based Coherence

Cache Controller

why is Inv possible in IS^D but not IM^{AD}?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Only sharers receive Inv;
dir must have received GetS

Directory-Based Coherence

Cache Controller

why impossible?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Directory-Based Coherence

Cache Controller

why impossible?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS ^D	GetM to dir → IM ^{AD}										
IS ^D	stall	stall	stall			stall		→ S		→ S		
IM ^{AD}	stall	stall	stall	stall	stall			→ M	→ IM ^A	→ M	ack-- → IM ^{AD}	
IM ^A	stall	stall	stall	stall	stall						ack-- → IM ^A	→ M
S	hit → S	GetM to dir → SM ^{AD}	PutS to dir → SI ^A			Inv-Ack to req → I						
SM ^{AD}	hit → SM ^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM ^{AD}		→ M	→ SM ^A		ack-- → SM ^{AD}	
SM ^A	hit → SM ^A	stall	stall	stall	stall						ack-- → SM ^A	→ M
M	hit → M	hit → M	PutM to dir → MI ^A	Data to req+dir → S	Data to req → I							
MI ^A	stall	stall	stall	Data to req+dir → SI ^A	Data to req → II ^A		→ I					
SI ^A	stall	stall	stall			Inv-Ack to req → II ^A	→ I					
II ^A	stall	stall	stall				→ I					

Last stable state was S;
no owner while in S

Directory-Based Coherence

Cache Controller

does dir actually need to send Data when in SM^{AD} ?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS^D	GetM to dir → IM^{AD}										
IS^D	stall	stall	stall			stall		→ S		→ S		
IM^{AD}	stall	stall	stall	stall	stall			→ M	→ IM^A	→ M	ack-- → IM^{AD}	
IM^A	stall	stall	stall	stall	stall						ack-- → IM^A	→ M
S	hit → S	GetM to dir → SM^{AD}	PutS to dir → SI^A			Inv-Ack to req → I						
SM^{AD}	hit → SM^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM^{AD}		→ M	→ SM^A		ack-- → SM^{AD}	
SM^A	hit → SM^A	stall	stall	stall	stall						ack-- → SM^A	→ M
M	hit → M	hit → M	PutM to dir → MI^A	Data to req+dir → S	Data to req → I							
MI^A	stall	stall	stall	Data to req+dir → SI^A	Data to req → II^A		→ I					
SI^A	stall	stall	stall			Inv-Ack to req → II^A	→ I					
II^A	stall	stall	stall				→ I					

Directory-Based Coherence

Cache Controller

does dir actually need to send Data when in SM^{AD} ?

Current state	Load	Store	Evict	Fwd-GetS	Fwd-GetM	Inv	Put-Ack	Data from dir [ack=0]	Data from dir [ack>0]	Data from owner	Inv-Ack	Inv-Ack (last)
I	GetS to dir → IS^D	GetM to dir → IM^{AD}										
IS^D	stall	stall	stall			stall		→ S		→ S		
IM^{AD}	stall	stall	stall	stall	stall			→ M	→ IM^A	→ M	ack-- → IM^{AD}	
IM^A	stall	stall	stall	stall	stall						ack-- → IM^A	→ M
S	hit → S	GetM to dir → SM^{AD}	PutS to dir → SI^A			Inv-Ack to req → I						
SM^{AD}	hit → SM^{AD}	stall	stall	stall	stall	Inv-Ack to req → IM^{AD}		→ M	→ SM^A		ack-- → SM^{AD}	
SM^A	hit → SM^A	stall	stall	stall	stall						ack-- → SM^A	→ M
M	hit → M	hit → M	PutM to dir → MI^A	Data to req+dir → S	Data to req → I							
MI^A	stall	stall	stall	Data to req+dir → SI^A	Data to req → II^A		→ I					
SI^A	stall	stall	stall			Inv-Ack to req → II^A	→ I					
II^A	stall	stall	stall				→ I					

No, already up-to-date copy
(load hits are allowed)

Directory-Based Coherence

Directory Controller

Current State	Event						
	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
I	Data to req → S	Data to req → M	Put-Ack to req → I	Put-Ack to req → I		Put-Ack to req → I	
S	Data to req → S	Data to req Inv to sharers → M	Put-Ack to req → S	Put-Ack to req → I		Put-Ack to req → S	
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	
S ^D	stall	stall	Put-Ack to req → S ^D	Put-Ack to req → S ^D		Put-Ack to req → S ^D	Data to mem → S

Directory-Based Coherence

how is it possible to receive PutM (not owner)
while in M state?

Directory Controller

Current State	Event						
	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
I	Data to req → S	Data to req → M	Put-Ack to req → I	Put-Ack to req → I		Put-Ack to req → I	
S	Data to req → S	Data to req Inv to sharers → M	Put-Ack to req → S	Put-Ack to req → I		Put-Ack to req → S	
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	
S ^D	stall	stall	Put-Ack to req → S ^D	Put-Ack to req → S ^D		Put-Ack to req → S ^D	Data to mem → S



Directory-Based Coherence

how is it possible to receive PutM (not owner)
while in M state?

Directory Controller

Current State	Event						
	GetS	GetM	PutS (not last)	PutS (last)	PutM (owner)	PutM (not owner)	Data
I	Data to req → S	Data to req → M	Put-Ack to req → I	Put-Ack to req → I		Put-Ack to req → I	
S	Data to req → S	Data to req Inv to sharers → M	Put-Ack to req → S	Put-Ack to req → I		Put-Ack to req → S	
M	Fwd-GetS to owner → S ^D	Fwd-GetM to owner → M	Put-Ack to req → M	Put-Ack to req → M	Data to mem Put-Ack to req → I	Put-Ack to req → M	
S ^D	stall	stall	Put-Ack to req → S ^D	Put-Ack to req → S ^D		Put-Ack to req → S ^D	Data to mem → S

GetM of a new cache arrives just before
PutM of original owner